

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №5

Выполнил:

Максимова Ольга

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. подключить и настроить rabbitMQ/kafka;
2. реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

Kafka была подключена и настроена с помощью docker-compose:

```
services:
  >Run Service
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.1
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  >Run Service
  kafka:
    image: confluentinc/cp-kafka:7.5.1
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    depends_on:
      - zookeeper
    healthcheck:
      test: ["CMD-SHELL", "kafka-broker-api-versions --bootstrap-server localhost:9092 | grep -q 'ApiVersion' || exit 1"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
```

```
>Run Service
kafka-init:
  image: confluentinc/cp-kafka:7.5.1
  depends_on:
    kafka:
      condition: service_healthy
  command: >
  sh -c '
    kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic rent.events --partitions 1 --replication-factor 1 &&
    kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic payment.events --partitions 1 --replication-factor 1 &&
    kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic accommodation.events --partitions 1 --replication-factor 1 &&
    kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic user.events --partitions 1 --replication-factor 1 &&
    kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic message.events --partitions 1 --replication-factor 1
  '
  restart: "no"
```

zookeeper запускает контейнер confluentinc/cp-zookeeper:7.5.1, пробрасывает порт 2181 и задает два параметра: клиентский порт и tick time.

kafka запускает брокер confluentinc/cp-kafka:7.5.1, который подключается к Zookeeper через zookeeper:2181. KAFKA_LISTENERS говорит, на каком адресе и порту брокер реально слушает внутри контейнера, а KAFKA_ADVERTISED_LISTENERS – какой адрес Kafka будет отдавать клиентам в metadata.

KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false" отключает автоматическое создание тем, чтобы темы создавались только явно.

healthcheck проверяет, что брокер уже отвечает, а depends_on указывает, что Kafka стартует после Zookeeper. depends_on управляет порядком запуска, а healthcheck позволяет дождаться реальной готовности сервиса, а не просто факта запуска контейнера.

kafka-init – это одноразовый контейнер, который ждет, пока Kafka станет healthy, и затем создает топики rent.events, payment.events, accommodation.events, user.events, message.events. Команда kafka-topics --create --if-not-exists делает так, что если тема уже есть, контейнер не упадет на повторном запуске. После выполнения этих команд контейнер завершится, потому что restart: "no" запрещает ему перезапускаться.

Также были настроены консьюмеры и продюсеры. Например:

```

import { subscribe, publish, Topics } from "common";
import type {
  AccommodationBlocked,
  PaymentProcessed,
  PaymentFailed,
} from "common";
import { rentService } from "../services";

export default async function setupRentListeners() {
  await subscribe<AccommodationBlocked>(Topics.Accommodation, async (msg) => {
    if (msg.eventType === "accommodation.blocked") {
      await rentService.markAsPendingPayment(msg.rentId);
    }
  });

  await subscribe<PaymentProcessed>(Topics.Payment, async (msg) => {
    if (msg.eventType === "payment.processed") {
      await rentService.handlePaymentCaptured(msg.rentId);
      await publish(Topics.Rent, {
        eventType: "rent.confirmed",
        rentId: msg.rentId,
        timestamp: Date.now(),
      });
    }
  });

  await subscribe<PaymentFailed>(Topics.Payment, async (msg) => {
    if (msg.eventType === "payment.failed") {
      await rentService.handlePaymentFailure(msg.rentId, msg.reason);
      await publish(Topics.Rent, {
        eventType: "rent.failed",
        rentId: msg.rentId,
        step: "payment_hold",
        status: msg.reason,
        timestamp: Date.now(),
      });
    }
  });
}

```

```

await initKafka("rent-service", "rent-service-group");
console.log("Kafka has been initialized!");

await initListeners();
console.log("Listeners have been initialized!");

await startKafkaConsumer();
console.log("Kafka consumer started!");

```

В итоге микросервисы могут публиковать события в свои топики и быть подписаны на события других микросервисов.

```

olgamaksimova@MacBook-Pro-Olga lab2 % docker exec -it kafka kafka-console-consumer \
  --bootstrap-server kafka:9092 \
  --topic user.events
{"eventType":"user.registered","email":"olga3@example.com","timestamp":1777889791303}
{"eventType":"user.loggedin","userId":"22c4bd8d-bee8-40bd-8d3c-148734f02669","email":"olga3@example.com","timestamp":1777889799556}
{"eventType":"user.verified","userId":"22c4bd8d-bee8-40bd-8d3c-148734f02669","isVerified":true,"role":"user","timestamp":1777889803859}

```

```
olgamaksimova@MacBook-Pro-Olga lab2 % docker exec -it kafka kafka-console-consumer \
--bootstrap-server kafka:9092 \
--topic rent.events

{"eventType":"rent.confirmed","rentId":"cecf22b3-6bc4-4e03-a90b-35ee457caf82","timestamp":1777889978609}
{"eventType":"rent.created","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","tenantId":"22c4bd8d-bee8-40bd-8d3c-148734f02669","landlordId":"6d6e0b08-6a9e-4138-bf23-66c9cc16d9b1","accommodationId":"8e20bbb2-7370-4ba4-974f-26bd383de16b","startDate":"2014-04-29","endDate":"2014-05-10","amount":50000,"timestamp":1777890073107}
{"eventType":"rent.confirmed","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","timestamp":1777890092408}
```

```
olgamaksimova@MacBook-Pro-Olga lab2 % docker exec -it kafka kafka-console-consumer \
--bootstrap-server kafka:9092 \
--topic accommodation.events

{"eventType":"accommodation.rented","rentId":"cecf22b3-6bc4-4e03-a90b-35ee457caf82","timestamp":1777889978674}
{"eventType":"accommodation.blocked","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","accommodationId":"8e20bbb2-7370-4ba4-974f-26bd383de16b","startDate":"2014-04-29","endDate":"2014-05-10","timestamp":1777890073176}
{"eventType":"accommodation.rented","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","timestamp":1777890092420}
```

```
olgamaksimova@MacBook-Pro-Olga lab2 % docker exec -it kafka kafka-console-consumer \
--bootstrap-server kafka:9092 \
--topic payment.events

{"eventType":"payment.created","paymentId":"1a7db4b4-6289-4c43-9e5c-060f8a9cb706","rentId":"cecf22b3-6bc4-4e03-a90b-35ee457caf82","amount":50000,"timestamp":1777889973675}
{"eventType":"payment.processed","paymentId":"1a7db4b4-6289-4c43-9e5c-060f8a9cb706","rentId":"cecf22b3-6bc4-4e03-a90b-35ee457caf82","amount":50000.00,"status":"captured","timestamp":1777889978508}
{"eventType":"payment.created","paymentId":"b5ac8687-0d6a-48bc-8f18-2a0be5a1da1f","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","amount":50000,"timestamp":1777890089436}
{"eventType":"payment.processed","paymentId":"b5ac8687-0d6a-48bc-8f18-2a0be5a1da1f","rentId":"3ad8e4f2-5f29-4df4-9139-acaca011b1ed","amount":50000.00,"status":"captured","timestamp":1777890092333}
```

```
olgamaksimova@MacBook-Pro-Olga lab2 % docker exec -it kafka kafka-console-consumer \
--bootstrap-server kafka:9092 \
--topic message.events

{"eventType":"message.sent","messageId":"ed56a65d-c492-455c-9d76-11662b0da183","fromUserId":"22c4bd8d-bee8-40bd-8d3c-148734f02669","toUserId":"6d6e0b08-6a9e-4138-bf23-66c9cc16d9b1","content":"Hello, how are you?","accommodationId":"8e20bbb2-7370-4ba4-974f-26bd383de16b","timestamp":177789011682}
```

Вывод

Было реализовано межсервисное взаимодействие посредством kafka.